11/02/26

Network Security 2, Rubric 3

Network Protocol & Traffic Analysis
The project must include packet capture and basic analysis using Wireshark with identification of at least 3 common protocols. A basic vulnerability assessment of one network protocol must be performed with documentation of findings. Network performance metrics must be collected and interpreted from a security perspective. Basic traffic pattern analysis must be performed to distinguish between normal and suspicious traffic. All analysis must include screenshots and simple explanations.

1. Lab Setup for Analysis

To capture clean traffic, you will use **Wireshark** directly on your Ubuntu VM or run a packet capture via the OPNsense GUI (**Interfaces > Diagnostics > Packet Capture**).

---

2. Required Project Deliverables & Analysis

🔹 **Component 1: Packet Capture & Protocol Identification**

Goal: Capture baseline network traffic and isolate three distinct standard protocols.

- **Execution Steps:**
  - Open Wireshark on your **Ubuntu VM** and start capturing on the main interface (`eth0` / `ens33`).
  - Open a browser and visit a webpage (generates **DNS** and **TLS/HTTPS** traffic).
  - Open a terminal and run `sudo dhclient -v` or restart your network interface (generates **DHCP** traffic).

    - Stop the capture and use the Wireshark filter bar to isolate each protocol.
- **Analysis Documentation Template:**
    - **Protocol 1: DNS (Domain Name System - UDP Port 53)**
        - *Explanation:* Used to resolve domain names to IP addresses. The capture shows a `Standard query A` request from Ubuntu to the OPNsense gateway (`192.168.10.1`).
    - **Protocol 2: TLS v1.3 (Transport Layer Security - TCP Port 443)**
        - *Explanation:* Provides encrypted web communication. The capture shows a `Client Hello` handshake initiating a secure session to the web server.
    - **Protocol 3: DHCP (Dynamic Host Configuration Protocol - UDP Ports 67/68)**
        - *Explanation:* Dynamically assigns network configurations. The capture isolates the `DHCP Request` and `DHCP ACK` transaction frames.
- 📸 **Screenshot Focus:** A combined or split screenshot of Wireshark showing the filtered rows for `dns`, `tls`, and `dhcp` highlighted in the packet list pane.

🔹 **Component 2: Basic Vulnerability Assessment of One Network Protocol**

Goal: Document the inherent security risks of using an unencrypted legacy protocol.

- **Target Protocol for Assessment: HTTP (Hypertext Transfer Protocol - TCP Port 80)**
- **Execution Steps:**
    1. Set up a simple unencrypted login form or basic site on your Server VM (VLAN 20).
    2. Start a Wireshark capture. Log into the site from your Ubuntu VM using a dummy credential (e.g., User: `admin`, Pass: `P@ssword123`).
    3. Stop the capture. Right-click any HTTP packet in Wireshark and select **Follow > TCP Stream**.
- **Vulnerability Assessment Findings:**
  text

```
None

[VULNERABILITY REPORT: CLEAR-TEXT PROTOCOL EXPOSURE]
```

```
- Protocol Assessed: HTTP / Version 1.1
- Risk Level: HIGH
- Vulnerability Description: HTTP transmits payload data completely
unencrypted.
  Any adversary positioned inline (Man-in-the-Middle) can reconstruct
the stream.
- Evidence: The full POST request body, including the plaintext
'username' and
  'password' string variables, was successfully intercepted and read
via raw packet assembly.
- Remediation: Enforce TLS (HTTPS) on OPNsense, redirecting port 80
traffic to port 443.
```

-
  Use code with caution.
- 📸 **Screenshot Focus:** The Wireshark **TCP Stream window** clearly showing the plaintext user credentials exposed in red and blue text blocks.

🔹 Component 3: Network Performance Metrics Collection

Goal: Gather network metrics and interpret how a security anomaly impacts overall system availability.

- **Execution Steps:**
  - Collect standard baseline performance statistics. In Wireshark, navigate to **Statistics > Conversations** and **Statistics > I/O Graphs**.
  - Run a baseline speed or throughput test between Ubuntu and the Server using `iperf3`.
- **Security Interpretation Metrics:**
  - **Throughput / Bandwidth:** Baseline data moves efficiently at line rate (e.g., 940 Mbps over virtual links). A sudden drop during an incident indicates bandwidth exhaustion (DDoS).
  - **Packet Loss Rate:** Normal operational baseline is `0%`. High packet loss indicates a flooded network queue or an active network jamming/degradation attack.
  - **Latency / RTT (Round Trip Time):** Normal RTT between subnets across the OPNsense firewall is `<2ms`. Spikes in latency point to high firewall CPU utilization due to heavy state table processing.

- 📸 **Screenshot Focus:** The Wireshark **I/O Graph** displaying a steady, flat baseline curve representing standard, healthy bandwidth usage.

🔹 **Component 4: Traffic Pattern Analysis (Normal vs. Suspicious)**

Goal: Contrast predictable corporate traffic patterns against hostile network reconnaissance.

- **Execution Steps:**
  1. **Normal Pattern:** Capture traffic while browsing a webpage normally from Ubuntu.
  2. **Suspicious Pattern:** Go to your **Kali Linux VM** and fire an aggressive scan at the server zone: `nmap -T4 -A 192.168.20.100`. Capture this footprint.
- **Direct Traffic Comparison:**

Traffic Attribute	Normal Pattern (Ubuntu Host)	Suspicious Pattern (Kali Attack Host)
Connection Flow	Orderly 3-way handshakes (SYN → SYN-ACK → ACK) followed by data exchange.	Massive bursts of rapid `SYN` packets sent across thousands of sequential ports.
Flag Distribution	Standard `ACK`, `PUSH-ACK`, and `FIN-ACK` flags tear down connections cleanly.	Excessive `RST` (Reset) flags or anomalous combinations (like NULL or FIN scans).
Packet Volume	Sustainable packet rate matching human interaction speed.	Exponential, vertical spike in the packet-per-second graph over a short duration.

- 📸 **Screenshot Focus:** A side-by-side view of the Wireshark packet pane. Left side: clean, blue HTTP/TCP handshakes. Right side: a continuous wall of dark red/black `TCP Spurious Retransmission` or `TCP RST` packets caused by the rapid Nmap scanning engine.
